

ActInf Textbook Group ~ Cohort 8 ~ Session 16

(Chapter 8) ~ 9/18/2025

Source: [YouTube](#) Playlist: Cohort 9 ~ Active Inference 2022 Parr et al. Textbook

Duration: 52:49 | Uploaded: Unknown Transcript method: auto_caption

All right, greetings everyone. Welcome slashback. It's September 18th and we're in chapter 8. So to begin, is there anything that anyone wants to bring up? a question from the KOD or a section from the textbook could be in chapter 8 or just anything that's top of mind if you want to start there. Just go for it or I'll I'll look in the chat. Um could you uh some context to the sort of philosophy of science uh perspective on uh continuous versus discrete modeling and how uh maybe other uh fields before active in inference um have tried to approach this question.

>> Nice. Yes. Well, obviously a huge question. It's a lot like the complementarity between analog and digital. Like if you have a cassette tape or a vinyl record, no matter how much you zoom in, there's always going to be some continuous value to the signal. So like a radio signal through the air or a physical media like tape or vinyl. Whereas digital records they have a sampling rate. So they are like very rapidly spaced samples you know asterisk asterisk pending zooming in to very high levels. So just at a first pass. Um, so when it comes to modeling time, which as the quote that starts the chapter hints at, time can be modeled as change. And in chapter 7 and in chapter 8, we're introduced to the two big ways that time is modeled, which is in chapter 7, in the discrete time case, you have discrete time steps could be very high frequency or could be big gaps between them relatively. And then you define transition functions or transition matrices which are iteratively applied. And so that's the B matrix or here number three on the top of figure 4.3. But that's a transition matrix that gets applied. So time step one you multiply by matrix you get to time step two. There's nothing in between one and two. There's just a rule that moves you forward one click at a time. There's no time step 1.1. In contrast, in the continuous time setting, it's more like you have a string and that string is going to be this continuous differentiable function through time. So that has different pros and cons which definitely we should explore and discuss more. But the big difference is continuous time models are like a path or a string through time. So they're continuous and unbroken and differentiable everywhere. In contrast, discrete models only have

values at specific intervals. And there are pros and cons to handling each of those. Which is why chapter 8 where we get to by the end is what about a hierarchical mixed generative model where you have a continuous time model on the lower level and a discrete model on the higher level. like we will we will get there today and talk about why that architecture gets converged upon. But this is kind of like the the whether it's about time being continuous or discrete or whether it's about space being continuous or discrete. That's definitely the ongoing debate. And then when it comes to our digital computers, even the continuous time models, you're going to have some digitized interval that's going to be used. And does this depend on the the perception or the action steps being continuous or discrete because you know they can be mixed and matched but it seems like it depends on the input uh classification. >> Yes. Um so seven and eight focus on discrete and continuous time settings. But then there's also separately from how time is handled. You could have discrete and continuous state spaces for like a variable and it could be the same underlying aspect of the world like temperature in the room. That could be a continuous variable between 0 and 100 or it could be just the integers 0 to 100 or the thermometer reading could be continuous or it could be constrained to be discrete or even the the agent's belief about temperature or what actions it's going to do. It could be um what angle will my elbow be at? That could be a continuous value or it could be I'm going to have my arm at the low, medium or high discrete position. So it's kind of like a 2x two matrix. Discrete and continuous time handling and then the state space that's under time evolution could be discrete or continuous. And there's kind of a natural affinity for discrete state spaces in discrete time. That's like kind of the the grid world, like the computer science fantasy is just like this discrete chessboard undergoing rule-based evolution. Like chess is a exact example of the discrete discrete pieces exist at discrete locations at discrete times. And then there's kind of this natural affinity of the continuous continuous where you just have like fields and flows and everything is nicely differentiable in state space and in time. But you also could have a discrete variable in continuous time. So that would look like here's an example of a continuous time linear system. And there's a there's a latent variable that's going to switch between mode one and mode two. And basically mode one is a low variability setting. Mode two is a high variability setting. So at all time steps it's like it's in mode one and it's outputting a data scatter and then that flips into mode two and you go into a high variance mode. That's that's a latent discrete variable switching in continuous time. And then of course the other fourth case would be you have a continuously valued number in discrete time. So that would be like you're using PIMDP and at every time step the agent is going to evaluate how safe is the situation and it's

like and then the next time step it's 655. So it's it's it's a decimal number but it's being discretely updated in time steps. So that's the kind of like lay of the lands of chapter seven and eight which and the reason why it makes sense to have those two is time is the special variable handling temperature as a continuous number or a set of ranked integers or high medium low those are all kind of like ordered sets. without the continuous ordered set is infinite and then the integer one is finite and then the one with high medium low is is like very low number of finite state possibilities but they're all kind of like there's a lower to a higher continuum. It's just a question of whether you have a smooth gradient, like does the volume knob have a smooth slider or does it have clicks, but that's roughly similar. And the big question for for the formalism, let alone how it gets implemented on on computers, is the difference is in the time handling. And then figure 4.3 is kind of like the Rosetta Stone figure because it lays out the structural isomorphism between the chapter 7 discretet time and chapter 8 continuous time approaches very blatantly laying them out to show their architecture all the time. I know that >> they had a few rain few too many rain. >> So they have a structural affinity, but then that's like kind of the provocative nature of figure 4.3 is like these two models, they do different things. It's like here's the here's this the MP3 file and here's the vinyl, but it's the same song. But they're totally different in how they get played out. But they they have enough overlap where you can kind of do the magic eye thing. If they were side by side, do the magic eye thing and like see how there are parts that play identical or analogous roles. And so that gets brought up in chapter 4. And then chapter seven and eight is kind of like the divide and conquer. And then it gets stitched back together in chapter nine with the empirical data part. But seven and eight focus on these two pieces. But anyone else just write a question or go for it. Yes, please. Carrie, >> um I just need to get this sort of question out of my mind. Um when I first learned programming and setting up models, um uh the amount of computing power was always a major constraint in setting up models that that we were always sort of forcing regardless of whatever we were observing into simpler formations in order to match the amount of computational power we So, as time has gone on and we have more computational power with computing, um, and this question like I I like your example of of analog versus digital. Um and again just to get me through this chapter and this so so what's being presented in this book is discrete and continuous time and the conversation about what would be sort of the optimal model for the conditions we're modeling um and whether it's more appropriate to use discrete or continuous. are are those models generated without the constraint of computing power and then we then we kind of constrain them after the fact or do we set up our models

based on the fact that we know that we're going to be constrained by our data? >> Yeah, that's that's an awesome question. So the the mo one mode is to write out the model in the way that's going to be the um best map representation and then take ameliating steps to deal if there is a computational challenge. For simple models, it's unlikely to come up even on a personal computer, but there are certain situations where computational costs could increase. And so this is sort of the computational complexity estimation area, which is like if every time step takes a certain amount of compute, then you get a linear increase in compute with a number of time steps. If every time step deeper that planning is happening, you get an exponential increase in the number of policies to consider. Then you have an exponential increase in computational complexity. If you're running replicates, it's a linear increase in the number of replicates. So between changing features like how many time steps is the simulation going to run, how many replicates of of parallel um execution to run. Um those are some of the or then then there's kind of modifying things within the simulation like make the planning horizon shorter or make the state spaces smaller. Then there are even further moves that you can make like you can do sampling of policies rather than evaluate them all. So like there's so many situations there's no there's no rule rule but if you're going to go the kind of chapter six approach where you're writing out the state spaces of your variables and everything like that and then you can do profiling as you run the first implementations and it's like okay did that take a hundth of a second or did it take 10 minutes to run. Um, as the as the state spaces grow larger, there's going to be some more resources required in CPU cycles and in memory. And then the computational complexity question is basically does it grow linearly, does it grow exponentially up or does it grow kind of saturating up? And then again like if you were taking the model to be published or you wanted to do parameter sweeps, you could run like the small, medium, and large model for 10, 100, 1,000 time steps. And then you get like a heat map of how long it takes. And so then that makes it empirical um exactly what resources are required. But suffice to say, for small state spaces, like for matrices that are just several to tens large, if you're in that kind of a of a zone, then models will run almost instantaneously on today's hardware. And the main place where I mean the main resource intensive challenge on the inbound on the perceptual is if you're getting like a big data input like the difference between getting a a a binary signal versus getting a 4K video feed. So when there's um highdimensional input, a common approach is to use a neural network to map that highdimensional input to a lower dimensional state space. And that's kind of the approach of the variational autoencoder even outside of active inference. So that helps get to a more manageable state space on the

inbound And then usually the main challenge for resources and compute on the action side is when there's either a lot of actions to select from or when there's a lot of time steps of planning. And that's because pi the the policy if we have four options at each time point like we're in a grid world then we have four expected free energies to calculate but then if we wanted to do a planning horizon of two we'd have 16 to calculate and then if we had three we'd have 64 to calculate etc. So that's the exponential increase in the number of EF calculations. And this is one of the fun things is like you can through chapter 6 write out the state space of all of these variables. Like we're going to get an integer coming in. We're going to map the integer to high, medium, or low. And then we have turn on the air conditioner or don't. and we're gonna plan over two horizons. Like back of the envelope, you can you can sketch out exactly which calculations are going to happen. Like we're going to take that integer value and we're going to we're going to have some matrix that has 100 by 3 and then we're going to have that 3x3 and then there's going to be two slices in the B matrix. So it's like actually the computational complexity estimates are straightforward. It's just and that helps you get very quickly to where there are possible bottlenecks like the large observation spaces or the large action spaces or One of the key differences with the discrete time is like again thinking of chess as the paradigmatic discrete time setting that is where planning can be made the most explicit and that's where you get the exponential blow up. Whereas in continuous time there is not planning as such. This is uh the Taylor Swift series as it is now known. Um the the the way that the continuous time looks forward and back is with the Taylor series like approach which is basically successive approximations to the derivatives and just expanding out the the this thread. So, the plus side of the Taylor series based look ahead or or back is you get all the lines. You get infinite look ahead for free or just for for for this for one one uh cost. But the downside is you actually aren't considering counterfactuals and you need secondary metrics to assess how likely is your look ahead. How likely is the look ahead to be actually in a confidence interval? So the continuous models are like that's that's why they are um that's why they often are found lower in the hierarchy where they reflect continuous valued inputs like pressure or temperature and then continuous valued actions like effort or angle of a joint. Right. And then for discretized planning there's a discrete and that that um type of continuous on bottom discrete on top that is um it was live stream 46 and that was this kind of um exploration of of folk psychology and just they mapped that exact. See if there's a not of the generative model. They have that type of model and they discuss it. Um yeah. So summary, small state spaces unlikely to be an issue just like you can run a tiny language model

on a old computer. But if you want to have a huge language model, you need more resources. But if they're smaller models, you never have an issue. just just kind of reviewing it but raise your hand or just write in chat. So 8.2 two goes into this classic pair of equations which is the observations depend on a latent state plus a noise term. So that's like the EEG data or the fMRI data intensity at a given voxel. The actual empirical observation depends on some generating function of a latent space and measurement noise. And then how that latent state changes through time. $\dot{X}$ is a function of the state of the latent variable at that time and sort of a cause operator plus a noise term. So this is like the neuroimaging observation depends on the neural state. Change in the neural state depends on the neural state and that distinction between what is the empirical data and its generation and then what is the latent data giving rise to empirical observations and how it changes through time. That's like the minimal classic setting. Um 8.2 is very similar and just replaces this kind of generic causal model uh intervention intervening variable to explicitly be action. Okay. Then there's going to kind of stay with that theme of the empirical data being generated by G and the change in the latent state being generated by an app and give a special a special case a simple special case which is a point attractor. this is is basically when um x is going to be the the uh v is the set point here and x is like the location of something. So this could be like a a gravity well or spring. when x is less than d x goes positive and then when x is more than v it goes negative. So it's it's linear on either side here. that gets expanded a little bit with that same V minus X theme, but instead of now being basically like the absolute value around a point attractor, now it's a Newtonian mechanical Hook's law spring around an attractor. So it's like we have a slinky that's hanging down and then if you push it up it'll wobble but come back to its resting point. If you pulled it down it would wobble and come back to its resting point. So that is how the causal aspects of the system get encoded into the dynamics of the latent variable and then those are used to generate observations. 8.3 goes into dynamical systems and like stochastic or probabilistic dynamical systems, linear or nonlinear dynamical systems. There's a ton of of work in and textbooks and everything. But here's one reason why the continuous why this isn't a continuous chapter is because we're talking about systems where the discretization is inefficient. Like if there was um a three-dimensional linear system with a thousand options on each, you'd have a million state space and you know or just a billion. Um yeah, they say limited space in the book. We cannot do justice to the large body of work, but they show some pieces. So now this is going to be in two dimensions. This is the spring in two dimensions. So V is going to be the spring attractor and X is going to be the location of of a two-dimensional

joint. Um so V could be the top down the this is from chapter 5 invoking the spinal cord reflex. The descending prediction of the motor system sets the attractor location V and then through um error minimization of proprioceptive inputs coming in you get ballistic movement towards the set point through that spring-like dynamic. So this is kind of like that slinky and then the top down the descending mental prediction changes where the set point is and then the slinky adapts to where that settles in to where that set point is because of incoming proprioceptive signals and the imperative to reduce the error difference between the descending prediction of where the joint ought to be and the incoming observation of where the joint is. So that's like expectation fulfillment based motor behavior through predictive error resolving motor loop with descending motor predictions. And then like you could these μ subx could be coming from a discrete model or continuous model. So that could be like I'm gonna I'm gonna want my hand in place one and then two and then one and then three and then you'd see the arm move in those ways. Okay. 8 point box 8.1 is very fascinating about sensory attenuation and it kind of goes into a little bit more of this situation which is like when should there be a dominant influence of the top down prediction and when should there be more of a top down influence or bottom up influence of the measurements coming in. So it's like all the proprioception is saying I am sitting in a chair I am not moving and then the top down prediction goes I am moving. So if the bottom up is being attended to then the top down anticipation is going to get corrected in the face of evidence to the contrary. Hence no movement is executed. However, this is kind of like when you engage the clutch in an automatic or in a in a manual car or when the eyes happens, there's a transient attenuation of the incoming stimuli that kind of gives this brief opening for the top-down prediction to be enacted even though it is not being borne out by the proprioception. But then once but then then the clutch can get re-engaged once the movement has already begun. And this empirically happens in motor action and in in isades being a special case of of motor action. Then the chapter pivots to a classical ecology setting, the locka voltera dynamics. So here's uh a winnerless competition with plants, herbivores, and carnivores. So it's like kind of a food web, food chain situation. And there's three continuous functions that have relationships to each other. So this could be like three economic variables or this could be three hormone levels or three uh population levels of a of a ecosystem. And then these two panels are like the state space in three dimensions is is in a cube. And then this is like the shadow or the projection of the cube looking at just the plant versus the herbivore where it mostly follows this limit attractor or the herbivore versus the carnivore. So this is just showing although you're just

getting instantaneous relationships, you can get sequential and iterative and unfolding dynamics through that. like even if there was just um the relationships of two hormones or of three hormones, you still could get structured unfolding just from the fixed relationships of how those hormones work. Um, and two more behavioral examples of where the lockable terror model is used and and the reason why it's brought up is because it's often used to model the relationships of brain regions in their activation. So region A will activate B, B activates C, and then C represses A. So if they all start at that baseline, A would start to pump up B. B would start to pump up C. But then once C starts to get rolling, it would shut down A, which would ultimately feed back to shut down C. But then once C was off, A would come back online. And so that's the winnerless competition part. And uh the left example, it's it's sometimes unclear with the black and white images. So the papers are cited but there's a eyelink uh associative learning example and then there's a writing kind of a cursive writing example. box 82 is is uh again a kind of mathematical note and it it it's a little bit of a standalone note, but it has to do with how do you optimize the beliefs in continuous time and like the the integral is that's the whole thing with like the infinitesimal and is there a really infinitely small variable? Are there continuous functions or are they just kind of small trapezoids? Um okay then they pivot from the mockable terra co c co c co c co c co c co c co c co c co c co c co c co c co c co c co c co-ependent dynamical systems with with cyclic attractors or limit attractors into the chaotic system of the loren attractor whole cool story great work and discussion in the um stochastic chaos and mark all blankets uh paper from I guess 21. Yeah. Yesterday, four years in a day ago. just to show this approach works for doing inference even on a chaotic system where small changes lead to large outcome differences system. So this is exactly analogous to the here we have a threedimensional dynamical system and then we have two two-dimensional projections of it here. Same thing there's three dimensional systems time evolution. Little bit hard to tell which the three lines are but they are three. and then two projections of Y on X and Z on Y. And not Lauren's has been used in this bird song example where basically there's two birds that are that are jointly singing. And it's not exactly the level of open-endedness of uh natural language conversation of humans because both of the birds are aware of a song that basically needs to be upheld or they expect it to be upheld. And then it's like turn taking is reflected by being in one attractor where one bird is singing and then it but that represses its own singing eventually and then that flips over uh to the other bird singing which is um in in in bio that's that's sometimes called a repressillator. But we can see a similar kind of pattern, but it's a two two-step refracillator. Um that idea of the generalized

synchrony which is like it's not correlation. Correlation is an example of synchrony but correlation is lock step where two things are correlated when they're exactly moving in the same level. But if two things were um moving oppositely, they would have a negative correlation. Or if there was some other like a u mapping between two variables, the overall correlation might be zero because the best fit regression would go straight, but the mutual information between those two variables could be very high. So generalized synchrony is like a more generalized version of correlation that's based on mutual information. rather than linear regression slope 8.5 just says yes legos work like this you can plug in you could have a continuous continuous model you could have discrete discrete you can also have discrete on top and then continuous on the bottom So here the palm DP the chapter 7 on the top level is going to discretely determine which target is going to be set at for basically the spring So here it's uh an is secade look example where there's a discrete decision of where to look at discrete time points and then there's ballistic movements to in continuous time to look at where the the intention is to look. Um here's another small mathematical side note box about a Gaussian mixture model there the Gaussian is an amazing distribution. It turns out that any distribution, you know, asterisk, etc., can be approximated by a mixture of Gaussians, even if it has a bunch of different humps or a very nonlinear shape. Um, and so you can kind of compose Gaussians and overlay them on the same variable. That's a Gaussian mixture model. You can also have nested uh Gaussians and that's called a hierarchical Gaussian model. So because the Gaussian or the normal distribution is so well behaved statistically and it so well represents distributions with a central tendency. It's used all the time. And then they review some of the papers that have applied continuous time models to mainly human and mamalian settings. So this chapter reads the most like a stochastic dynamical systems text review. But it also just shows that's what's so cool about treating action as a variable in a statistical model is like variable two of these. This could be you know cortisol the sound level and your effort in flexing your quadriceps. Not exactly those three variables, but you could ask how are those variables through time related? And then the fact that two of those variables or you know the fact that one of the variable is an external variable, one of them is a sensemaking variable and one of them is an action variable. there's a unified handling of all of them versus artificially splitting the signal processing problem from the control theory problem. So it's a little bit of a read between the lines because the examples that come up are largely like okay we have a spring attractor we have a lock of terraen attractor and a nested model and then an application of a discrete continuous model. But it just speaks to how unified and and nonsellacious this is to

handle perception, cognition, and action in continuous and discrete time, which is where we get at the end of chapter 8. So they're a good pair of chapters. It revisits this kind of chapter 7, you know, chessboard, grid world, computer science, rule-based evolution, state transition concept. Um, and that gets kind of complemented with this more continuous dynamical differentiable perspective. And then in practice you can mix and match them. And one of the directions of the the field and tool development is to make it easier to compose those models. any just section of eight. Anyone wants to just or a thought or a question they had on it? Should we by default, Sylvia, start with a discrete time model? as sort of usual, it depends what you want to ask. Um, for example, in chapter 7 with the rat and the T-maze, like by modeling this as a series of discrete time steps, start location and then go to one of the locations or go to the queue, it's not saying, well, but you could have made a continuous model of the of the rat moving in space. So when there are like checkpoints or milestones, the discrete can be an enormous simplifier or like if there's like zones or phases. Like we're going to study the the probability of being hypoglycemic or normal blood sugar range and just like modeling the transition in or out of there. that is just going to be like a 2x two matrix. So that's going to be super simple and interpretable versus trying to mix a line to do that. So if you're dealing with something where the differences um between time points are discrete then yes most of the software resources are going to be discrete time discrete state spaces or continuous state spaces. But then if you um want to have an example where at every time point along a continuous line, you want to have a value that's changing. So, like, is it is the situation more like a chess game or is it more like you're tracking the location of of a floating thing in the pool? And then, okay, so it's more like a pool. And then what would be the pros and cons of overlaying the chessboard on the pool? Well, if we just overlay a one by one chessboard, it's going to be useless. If we overlay a 1 million by a million, that's where we get into that computational challenge and we lose interpretability and we're just calculating these large matrices. But if there's some discrete approximation to space andor time, it's very often done. and this is kind of a key buried line basically that continuous dynamical systems are a plausible continuous system that might underwrite the discrete sequential dynamics. So that's like again we have the vinyl record and we have the digital song. We could and it just um it's just a you know a note being a different note being played. If we were to overlay them, what we would like to see would be we have a line a continuous line and then exactly on the line are the discrete time points. So it's it's kind of like the the the beads on a necklace versus the actual string. And then just which mathematics or which software are going to make it easier

to work with just the beads locations versus the strings location. The discrete time you get the transition matrix that describes how states go into the next state, but you don't get derivatives in the same way. Okay, any last uh questions? Okay, awesome. I will